CS-781 Capstone, Phase 2: Legal Named Entity Recognition

Character-level legal NER on the four Phase-1 language models

Manish Kumar (251330604)
CS-781, IIT Kanpur

October 2026

Abstract

We tag twelve legal entity types (parties, counsel, judges, witnesses, authorities, courts, statutes, precedents, dates and case numbers) as character spans in long Indian judgments, starting from the four language models we pre-trained in Phase 1. A shared pipeline converts character spans to BIO token tags and back (oracle macro-F1 0.9945 with punctuation pre-splitting), runs each backbone as a token encoder, and decodes with a BIO-constrained Viterbi. Six submissions cover all four architectures and five adaptation methods: full fine-tuning (A3), LoRMA+ (A4), LoRA (A1), Houlsby adapters (A2), partial fine-tuning (A4) and knowledge distillation into a 4-block A3 student. A4 with LoRMA+ scores the highest dev strict macro-F1 (0.4857). A sweep of 37 one-change variants, an inference-only decoding sweep and a three-split confirmation follow; the best confirmed setting is E2 (A4 LoRMA, original backbone), prop10 (0.518±0.016). An error analysis shows that 37.9% of gold entities are not predicted at all. All numbers are generated from the run outputs by `p2/report/make_figures.py`.

Contents

1 Introduction **3**

2 Data **3**

3 From characters to tags and back **4**

4 Methods **4**
4.1 Backbones 4
4.2 Adaptation methods 5
4.3 Training 5
4.4 The six experiments 5

5 Results **6**
5.1 The six experiments 6
5.2 One-change sweep 7
5.3 Decoding and post-processing 8
5.4 Three-split confirmation 9
5.5 Learning curve 10
5.6 Error analysis 10
5.7 Cleaned Phase-1 backbones 11
5.8 Submissions 11

6 Analysis **11**

7 Limitations and next steps **11**

8 Conclusions **12**

9 Reproducibility **12**

1 Introduction

Legal named entity recognition (L-NER) identifies entities by their *legal role*: not just a person, but the appellant, the respondent, their counsel, a witness or the judge. Phase 2 asks for the character start and end offset and the label of every entity in a judgment; scoring is strict (span and label must both match exactly) and the primary metric is macro-F1 over the twelve labels. The rules require the Phase-1 models as the only pretrained weights, all four Phase-1 architectures across the submissions, at least five different fine-tuning or parameter-efficient techniques, and knowledge distillation; at least six submissions are needed.

The report follows the work in order: the data (Section 2), the character/token pipeline (Section 3), the backbones and adaptation methods (Section 4), the six experiments (Section 5.1), the tuning sweep, decoding sweep and three-split confirmation (Section 5.2 to Section 5.4), the learning curve (Section 5.5), an error analysis (Section 5.6) and the effect of the cleaned Phase-1 backbones (Section 5.7).

2 Data

- **Training**: 70 documents (two folds of 35) with gold spans as `[start, end, label, text]`; **test**: 35 documents without labels.
- Documents are long: median 4,488 words, maximum 19,978 words, against a 1,024-token Phase-1 context.
- 8,393 gold entities over twelve labels (Table 1); counsel, judges and witnesses have fewer than 250 examples each.
- **Dev split**: 10 of the 70 documents (`data.split_seed`=0) are held out for model selection and every comparison, the same for every run. For the confirmation (Section 5.4) two more disjoint-ish splits (seeds 1, 2) are used.

Table 1: Gold entities per label in the 70 training documents.

Label	Entities
APP	407
RESP	381
A.COUNSEL	195
R.COUNSEL	184
JUDGE	245
WIT	187
AUTH	1,003
COURT	1,013
STAT	1,260
PREC	1,183
DATE	1,575
CASENO	760
Total	8,393

3 From characters to tags and back

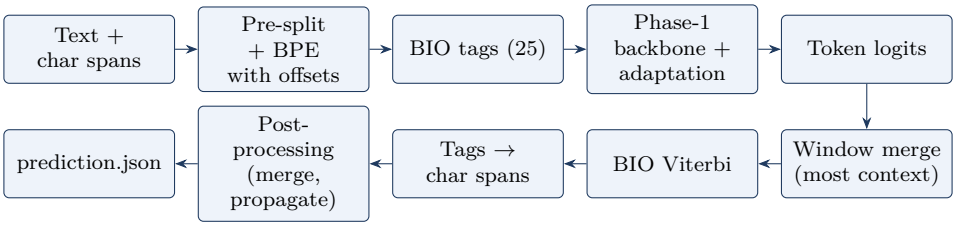


Figure 1: The L-NER pipeline (`lner/data.py`, `lner/engine.py`, `lner/decoding.py`).

- **Tokenisation with offsets.** The Phase-1 byte-level BPE (32k) encodes the text and returns the character offsets of every token, with the leading space of a token excluded.
- **Punctuation pre-splitting** (`data.presplit`). Legal text has runs such as `).` and `],`; an entity often ends in the middle of one, so BPE merges the entity's last character with the following punctuation. Cutting punctuation runs into separate pieces before BPE puts more entity boundaries on token boundaries.
- **BIO tags.** A token that overlaps a gold span gets `B-`/`I-`*label*, the rest `O`: 25 tags.
- **Oracle check** (`scripts/check_alignment.py`). Encoding the gold spans as tags and decoding them back gives the best score any tagger can reach with this tokenisation: macro-F1 0.9866 without pre-splitting and 0.9945 with it.
- **Windows.** Training uses 512-token windows with stride 256 and a random shift each epoch. Inference uses stride 128; each token keeps the logits of the window in which it has the most context on both sides.
- **Decoding.** Viterbi over the tag logits with transitions that break BIO (e.g. `O`→ `I-X`) forbidden; `argmax` is the unconstrained alternative. Tags are mapped back to character spans with trailing whitespace trimmed.
- **Post-processing** (inference only, Section 5.3). *merge*: two consecutive spans with the same label separated only by spaces are joined. *propagatek*: a string predicted as an entity (at least k characters) is also tagged, with its majority label, wherever else it appears as a whole word untagged in the same document.
- **Metrics** (`lner/metrics.py`). Strict macro precision/recall/F1 as on Codabench, plus micro-F1 and *span-only* F1 (boundaries right, label ignored) for error analysis.

4 Methods

4.1 Backbones

The Phase-1 model returns one hidden vector per token (`lner/backbones.py`); the language-model head is replaced by a token-classification head (linear by default; an MLP and a CRF head [6] were also tried).

- **A1 GPT** and **A4 efficient**: either the Phase-1 causal mask is kept (`attention: causal`, a token sees only its left context) or removed (`bidirectional`; A4's local layers then see ±256 tokens). The six experiments use bidirectional attention.
- **A3 prefix-LM**: the whole window is treated as the bidirectional prefix.
- **A2 encoder–decoder**: either the encoder output alone, or the decoder run over the same window while cross-attending to the encoder (`encdec_output: decoder`, BART-style tagging).

A2 diagnostic. Tagging from the A2 encoder output gave dev macro-F1 of only 0.11–0.15 (Table 3, "enc." rows). The mean cosine similarity between token vectors of the Phase-1 A2 encoder output is 0.92, against 0.19 for A3 (measured once, `p2/README.md`): the encoder output is nearly collapsed and the decoder does most of the language modelling. E4 therefore uses the decoder states.

4.2 Adaptation methods

All implementations are in `lner/peft.py`, written from the papers; the head is always trained.

- **Full fine-tuning**: every backbone weight.
- **Partial (layer-wise) fine-tuning**: only the top n blocks and the final norm (E5: top 4 of 10).
- **LoRA** [5]: a frozen linear layer W_0 gets an additive low-rank update, $h = W_0 x + \frac{\alpha}{r} BAx$ with $B = 0$ at initialisation.
- **LoRMA+** [2]: a multiplicative low-rank update, $W = (I + \frac{\alpha}{r} BA)\, W_0$, computed as $h_0 = W_0 x$, $h = h_0 + \frac{\alpha}{r} B(Ah_0)$ so its cost matches LoRA. **LoRMA$_\pi$** (sweep only) replaces $I + BA$ by a rank-inflating permutation of BA initialised to the identity.
- **Houlsby adapters** [4]: a bottleneck MLP ($d \to 64 \to d$, up-projection initialised to zero) after the attention and the MLP sub-layers; layer norms are also trained.
- **BitFit** [1] (sweep): only biases and norm gains.
- **IA^3** [8] (sweep): learned scaling of keys, values and the MLP's inner activation, initialised to one.
- **Knowledge distillation** [3] (E6): the student is A3 with 4 of its 8 blocks (blocks 1, 3, 5, 7; 52.9M parameters against 81.3M for the teacher), fully fine-tuned with $\mathcal{L} = \alpha\, \mathrm{CE} + (1 - \alpha)\, T^2\, \mathrm{KL}(p^T_{\text{teacher}} \,\|\, p^T_{\text{student}})$, $\alpha = 0.5$, $T = 2$, teacher E1 (A3 full fine-tuning).

LoRA-family methods wrap the attention projections (`q k v o`) and the MLP projections, with $r = 16$, $\alpha = 32$, dropout 0.05.

4.3 Training

AdamW, 15 epochs with early stopping after 5 epochs without dev improvement (best epoch kept), batch 16 windows, 10% linear warm-up then cosine decay, weight decay 0.01, gradient clipping 1.0. Learning rate 10^{-3} for LoRA/LoRMA/adapters, 2×10^{-4} for partial and KD, 10^{-4} for full fine-tuning; the head always uses 10^{-3}. The loss is token cross-entropy; the sweep also tries an `O`-tag weight, inverse-square-root class weights, focal loss [7], layer-wise LR decay, dropout and a CRF head. Only trainable parameters are saved, so PEFT checkpoints are a few MB.

4.4 The six experiments

Table 2: Submitted experiments (`p2/configs/`).

ID	Phase-1 model	Method	Config	Notes
E1	A3 prefix-LM	full fine-tuning	`e1_a3_full.yaml`	KD teacher
E2	A4 efficient	LoRMA+	`e2_a4_lorma.yaml`	mandatory LoRMA
E3	A1 GPT	LoRA	`e3_a1_lora.yaml`	
E4	A2 enc–dec	Houlsby adapters	`e4_a2_adapter.yaml`	decoder states
E5	A4 efficient	partial (top 4 blocks)	`e5_a4_partial.yaml`	
E6	A3, 4 of 8 blocks	KD from E1	`e6_kd_a3_small.yaml`	mandatory KD

5 Results

5.1 The six experiments

Table 3: Dev split 0 (10 documents). Span = span-only F1. Time in minutes; memory = peak GPU allocation.

Run	Arch	Method	Trainable	%	Macro-F1	P	R	Micro-F1	Span	Min	GB	MB
E1_a3_full	A3	full	81.30M	100.0	0.4267	0.447	0.441	0.507	0.560	2.1	3.7	325.2
E2_a4_lorma	A4	lorma	2.23M	2.5	0.4857	0.517	0.478	0.510	0.573	4.4	4.5	9.0
E3_a1_lora	A1	lora	1.79M	2.1	0.3564	0.395	0.356	0.451	0.503	2.8	3.2	7.2
E4_a2_adapter	A2	adapter	2.04M	2.2	0.3892	0.422	0.389	0.447	0.489	3.1	2.9	8.2
E5_a4_partial	A4	partial	25.19M	28.8	0.4697	0.488	0.474	0.520	0.559	1.4	2.3	100.8
E6_kd_a3_small	A3	full (KD)	52.95M	100.0	0.3443	0.372	0.344	0.434	0.483	1.4	2.6	211.8
diag_E4_a2_adapter_encoder	A2 (enc.)	adapter	1.02M	2.0	0.1116	0.155	0.094	0.186	0.186	1.5	1.4	4.1
diag_a2_partial_encoder	A2 (enc.)	partial	14.79M	29.6	0.1277	0.171	0.113	0.201	0.211	1.0	1.1	59.2
diag_a2_full	A2 (enc.)	full	50.04M	100.0	0.1493	0.171	0.147	0.226	0.265	0.7	2.0	200.2

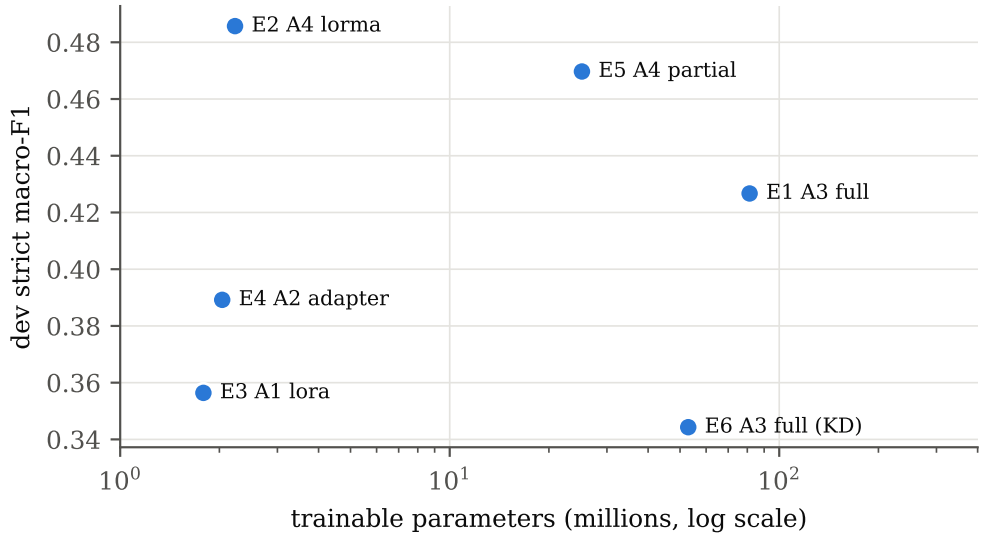


Figure 2: Dev macro-F1 against the number of trained parameters.

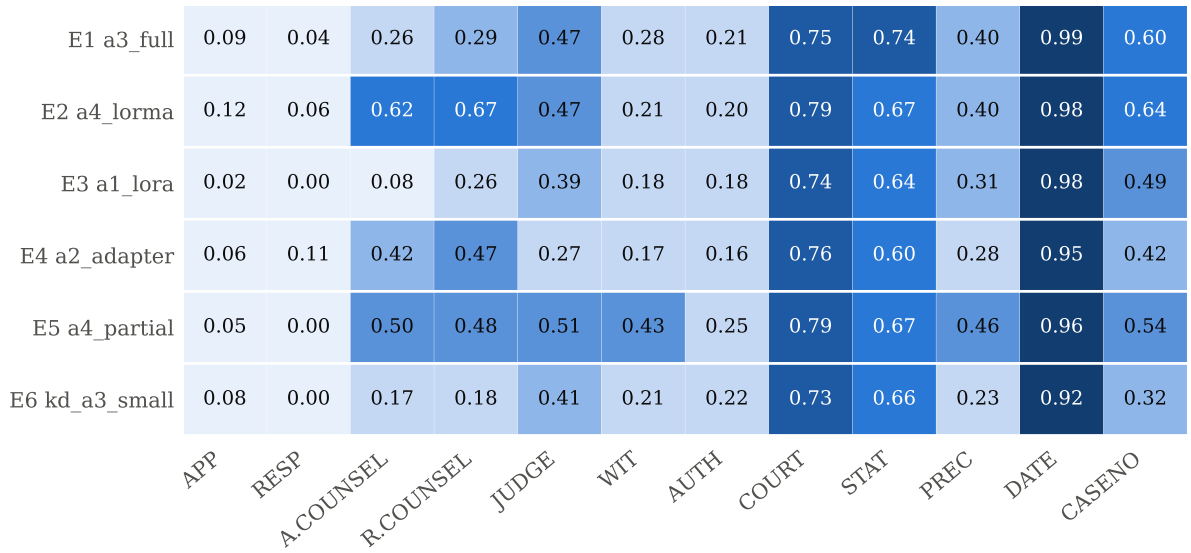


Figure 3: Dev F1 per label for the six experiments.

5.2 One-change sweep

Each variant changes one thing in one of the experiments (the "change" column is the difference between the two `config.yaml` files). All are scored on dev split 0, so a single difference of about ±0.02 is within the split-to-split variation measured in Section 5.4.

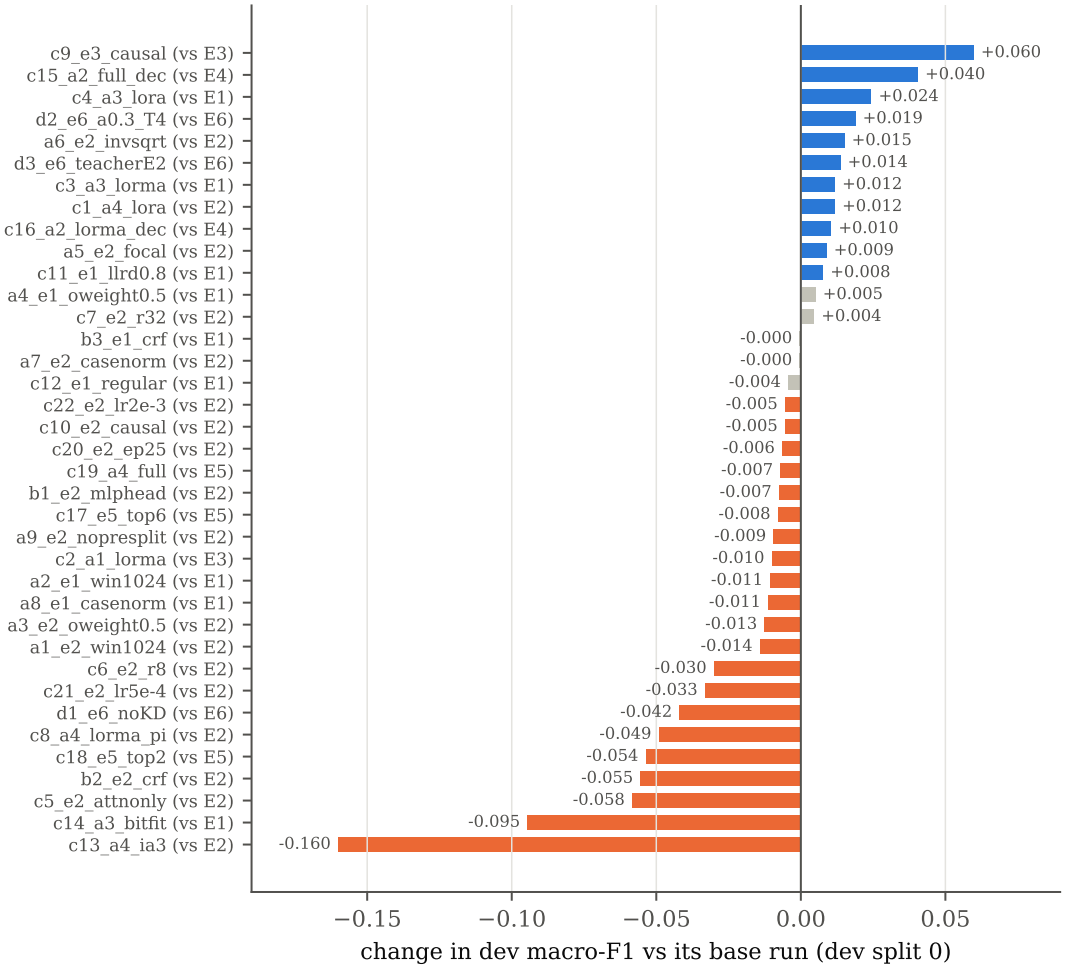


Figure 4: Change in dev macro-F1 of each variant against its base experiment (blue: $>+0.005$, orange: <-0.005, grey: in between).

Table 4: One-change sweep (`sweeps/RESULTS.md`).

Variant	Base	Change	Base F1	F1	Δ	Span
a6_e2_invsqrt	E2	label_weight=inv_sqrt	0.4857	0.5007	+0.0150	0.605
c1_a4_lora	E2	method=lora	0.4857	0.4974	+0.0117	0.605
a5_e2_focal	E2	focal_gamma=2.0, loss=focal	0.4857	0.4945	+0.0088	0.589
c7_e2_r32	E2	alpha=64, r=32	0.4857	0.4902	+0.0045	0.605
a7_e2_casenorm	E2	case_norm=True	0.4857	0.4853	-0.0004	0.580
c10_e2_causal	E2	attention=causal	0.4857	0.4803	-0.0054	0.562
c22_e2_lr2e-3	E2	lr=0.002	0.4857	0.4803	-0.0054	0.584
c20_e2_ep25	E2	epochs=25, patience=8	0.4857	0.4793	-0.0064	0.586
b1_e2_mlphead	E2	head=mlp	0.4857	0.4782	-0.0075	0.568
a9_e2_nopresplit	E2	presplit=False	0.4857	0.4763	-0.0094	0.555
a3_e2_oweight0.5	E2	o_weight=0.5	0.4857	0.4732	-0.0125	0.549
a1_e2_win1024	E2	window=1024, batch_size=8, stride=512, window=1024	0.4857	0.4716	-0.0141	0.570
c19_a4_full	E5	method=full, lr=0.0001	0.4697	0.4627	-0.0070	0.603
c17_e5_top6	E5	n_train_layers=6	0.4697	0.4618	-0.0079	0.572
c6_e2_r8	E2	alpha=16, r=8	0.4857	0.4557	-0.0300	0.561
c21_e2_lr5e-4	E2	lr=0.0005	0.4857	0.4526	-0.0331	0.542
c4_a3_lora	E1	alpha=32, method=lora, r=16, lr=0.001	0.4267	0.4510	+0.0243	0.576
c3_a3_lorma	E1	alpha=32, method=lorma, r=16, lr=0.001	0.4267	0.4384	+0.0117	0.559

Variant	Base	Change	Base F1	F1	Δ	Span
c8_a4_lorma_pi	E2	method=lorma_pi	0.4857	0.4366	-0.0491	0.577
c11_e1_llrd0.8	E1	llrd=0.8	0.4267	0.4343	+0.0076	0.563
a4_e1_oweight0.5	E1	o_weight=0.5	0.4267	0.4317	+0.0050	0.581
b2_e2_crf	E2	head=crf	0.4857	0.4303	-0.0554	0.580
c15_a2_full_dec	E4	method=full, lr=0.0001	0.3892	0.4296	+0.0404	0.513
c5_e2_attnonly	E2	targets=['q', 'k', 'v', 'o']	0.4857	0.4274	-0.0583	0.549
b3_e1_crf	E1	head=crf	0.4267	0.4263	-0.0004	0.604
c12_e1_regular	E1	dropout=0.2, weight_decay=0.1	0.4267	0.4224	-0.0043	0.575
a2_e1_win1024	E1	window=1024, batch_size=8, stride=512, window=1024	0.4267	0.4162	-0.0105	0.560
c9_e3_causal	E3	attention=causal	0.3564	0.4161	+0.0597	0.537
c18_e5_top2	E5	n_train_layers=2	0.4697	0.4161	-0.0536	0.522
a8_e1_casenorm	E1	case_norm=True	0.4267	0.4155	-0.0112	0.613
c16_a2_lorma_dec	E4	alpha=32, method=lorma, r=16	0.3892	0.3996	+0.0104	0.488
d2_e6_a0.3_T4	E6	alpha=0.3, temperature=4.0	0.3443	0.3631	+0.0188	0.490
d3_e6_teacherE2	E6	teacher_run=runs/E2_a4_lorma	0.3443	0.3579	+0.0136	0.519
c2_a1_lorma	E3	method=lorma	0.3564	0.3464	-0.0100	0.490
c14_a3_bitfit	E1	method=bitfit, lr=0.001	0.4267	0.3322	-0.0945	0.475
c13_a4_ia3	E2	method=ia3, lr=0.003	0.4857	0.3257	-0.1600	0.433
d1_e6_noKD	E6	alpha=None, teacher_run=None, temperature=None	0.3443	0.3024	-0.0419	0.447

5.3 Decoding and post-processing

The trained E1, E2 and E5 models are re-decoded with different inference strides, decoders and post-processing, without retraining (`scripts/infer_sweep.py`). For E2 at stride 128: argmax 0.3730, BIO Viterbi 0.4857, Viterbi + propagate5 0.5142.

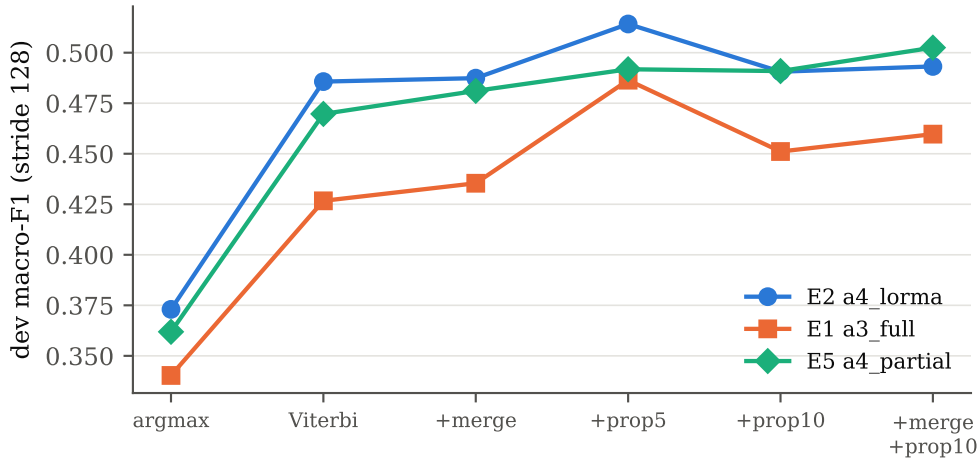


Figure 5: Dev macro-F1 for each decoding / post-processing option (stride 128).

Table 5: Inference-only sweep: macro-F1 at strides 64 / 128 / 256, and precision / recall at stride 128.

Run	Decoding	64	128	256	P	R
E2	argmax	0.3677	0.3730	0.3740	0.364	0.406
	viterbi	0.4831	0.4857	0.4816	0.517	0.478
	viterbi+merge_space	0.4844	0.4874	0.4838	0.523	0.475
	viterbi+propagate5	0.5181	0.5142	0.5111	0.498	0.554
	viterbi+propagate10	0.4867	0.4906	0.4855	0.503	0.502
	viterbi+merge+propagate10	0.4889	0.4932	0.4885	0.509	0.500
E1	argmax	0.3407	0.3403	0.3455	0.327	0.393
	viterbi	0.4261	0.4267	0.4259	0.447	0.441
	viterbi+merge_space	0.4346	0.4354	0.4333	0.459	0.445
	viterbi+propagate5	0.4863	0.4864	0.4848	0.456	0.558
	viterbi+propagate10	0.4509	0.4511	0.4506	0.437	0.502
	viterbi+merge+propagate10	0.4593	0.4597	0.4583	0.448	0.506
E5	argmax	0.3618	0.3619	0.3628	0.339	0.413
	viterbi	0.4649	0.4697	0.4649	0.488	0.474
	viterbi+merge_space	0.4776	0.4811	0.4782	0.503	0.479
	viterbi+propagate5	0.4909	0.4918	0.4978	0.465	0.535
	viterbi+propagate10	0.4927	0.4909	0.4982	0.472	0.524
	viterbi+merge+propagate10	0.5059	0.5025	0.5114	0.488	0.529

5.4 Three-split confirmation

The most promising settings were retrained on dev splits 0, 1 and 2 (`scripts/confirm.py`) and scored with every post-processing option.

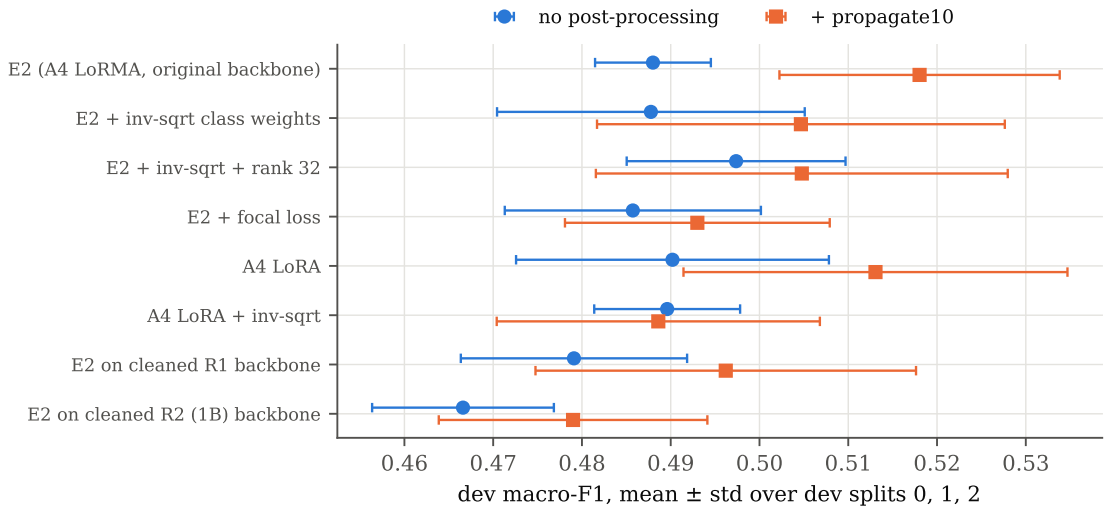


Figure 6: Mean ± standard deviation over three dev splits, without post-processing and with propagate10.

Table 6: Dev macro-F1, mean ± std over three dev splits, per post-processing option.

Setting	none	merge	prop5	prop10	merge+prop5	merge+prop10
E2 (A4 LoRMA, original backbone)	0.488±0.007	0.486±0.009	0.506±0.027	0.518±0.016	0.504±0.032	0.516±0.018
E2 + inv-sqrt class weights	0.488±0.017	0.485±0.015	0.490±0.025	0.505±0.023	0.486±0.023	0.501±0.021
E2 + inv-sqrt + rank 32	0.497±0.012	0.496±0.013	0.508±0.013	0.505±0.023	0.508±0.007	0.502±0.018
E2 + focal loss	0.486±0.014	0.485±0.009	0.498±0.029	0.493±0.015	0.498±0.024	0.492±0.013
A4 LoRA	0.490±0.018	0.485±0.014	0.517±0.043	0.513±0.022	0.513±0.042	0.508±0.021
A4 LoRA + inv-sqrt	0.490±0.008	0.490±0.006	0.480±0.016	0.489±0.018	0.481±0.013	0.489±0.018
E2 on cleaned R1 backbone	0.479±0.013	0.477±0.014	0.487±0.023	0.496±0.021	0.484±0.027	0.494±0.023
E2 on cleaned R2 (1B) backbone	0.467±0.010	0.465±0.011	0.462±0.020	0.479±0.015	0.461±0.021	0.478±0.016

5.5 Learning curve

E2 retrained on 15 and 30 of the 60 training documents of split 0 (`data.max_train_docs`): 0.339, 0.448 and 0.486 dev macro-F1 for 15, 30 and 60 documents.

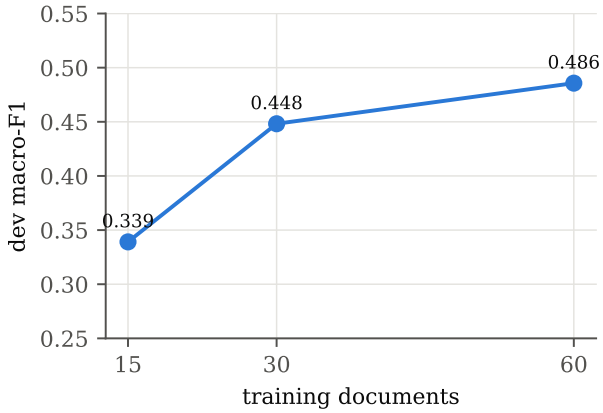


Figure 7: E2 dev macro-F1 against the number of training documents.

5.6 Error analysis

`scripts/error_analysis.py` matches every gold entity of E2's dev predictions by overlap: *exact* (true positive), *wrong label* (same span), *boundary* (overlapping prediction with the same label), *boundary + label*, or *missed* (no overlapping prediction). Over all labels: exact 45.7%, wrong label 5.6%, boundary 8.0%, boundary + label 2.9%, missed 37.9%; 195 predictions overlap no gold entity. Most frequent confusions (gold → predicted): APP→RESP 35, PREC→APP 11, PREC→RESP 10, WIT→PREC 9, WIT→RESP 7, APP→PREC 7, APP→AUTH 7, PREC→JUDGE 7.

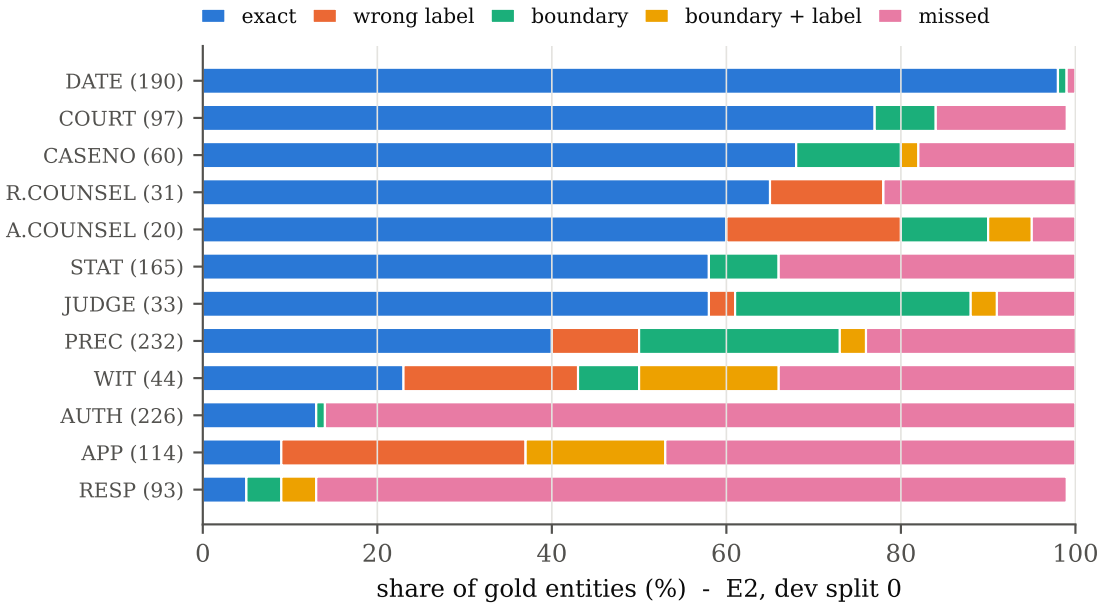


Figure 8: What happens to each gold entity of E2 on dev split 0, per label (gold count in brackets).

Table 7: Error breakdown per label (E2, dev split 0); spurious = predictions overlapping no gold entity.

Label	Gold	Exact	Wrong label	Boundary	Bound.+label	Missed	Spurious
APP	114	9%	28%	0%	16%	47%	29
RESP	93	5%	0%	4%	4%	86%	7
A.COUNSEL	20	60%	20%	10%	5%	5%	0
R.COUNSEL	31	65%	13%	0%	0%	23%	2
JUDGE	33	58%	3%	27%	3%	9%	11
WIT	44	23%	20%	7%	16%	34%	37
AUTH	226	13%	0%	1%	0%	86%	27
COURT	97	77%	0%	7%	0%	15%	10
STAT	165	58%	0%	8%	0%	34%	11
PREC	232	40%	10%	23%	3%	25%	44
DATE	190	98%	0%	1%	0%	1%	1
CASENO	60	68%	0%	12%	2%	18%	16

5.7 Cleaned Phase-1 backbones

After Phase 1 was retrained on a cleaned corpus (Phase-1 report), the E2 recipe was rerun unchanged on the new A4 checkpoints over the same three dev splits (Table 6, last two rows): original backbone 0.488±0.007, cleaned R1 (200M tokens) 0.479±0.013, cleaned R2 (1B tokens) 0.467±0.010 without post-processing; 0.518, 0.496 and 0.479 with propagate10. The submitted Phase-2 models use the original backbones.

5.8 Submissions

One Codabench zip per experiment, `prediction.json` at the root (`p2/submissions/`).

To be written by the author — leaderboard scores
Add the Codabench test macro-F1 of each submission here.

6 Analysis

To be written by the author — analysis of the results
The course rules do not allow LLM-written result analysis. Questions the course asks (§8.7, §20.1), with the evidence above:

- Which adaptation technique and which Phase-1 architecture work best for L-NER, and why (Table 3, Figure 2)?
- Can parameter-efficient methods approach full fine-tuning?
- Does distillation keep performance while reducing size?
- Which entity types are hard, and is the problem labels, boundaries or recall (Figure 3, Figure 8)?
- What did the sweep teach, given the split-to-split variation (Figure 4, Figure 6)?
- Why might a better Phase-1 language model not give better NER (Section 5.7)?
- What failed, and why (A2 encoder, CRF head, IA^3, BitFit)?

7 Limitations and next steps

Facts that limit the conclusions (the interpretation is the author's):

- Model selection and every comparison use 10 dev documents; most sweep differences are within the split-to-split standard deviation (0.007–0.043 in Table 6).

- One seed per run.
- Windows of 512 tokens against documents of median 4,488 words; party roles are stated in the header and must be resolved thousands of tokens later.
- 70 labelled documents; the learning curve (Figure 7) was still rising at 60.
- Post-processing is document-level but rule-based; it is not used during training.

To be written by the author — what you would do next
With more time or compute: ...

8 Conclusions

To be written by the author — conclusions
Summarise what you conclude from the experiments, in your own words.

9 Reproducibility

All six experiments + zips	`bash run_all.sh` (or `bash run_all.sh e3`)
One experiment	`python train.py -config configs/e2_a4_lorma.yaml`
Override a setting	`python train.py -config configs/e2_a4_lorma.yaml -set peft.r=8 run_name=E2_r8`
Predict / dev report	`python predict.py -run runs/E2_a4_lorma -split dev`
One-change sweep	`python scripts/sweep.py`
Decoding sweep	`python scripts/infer_sweep.py`
Three-split confirmation	`python scripts/confirm.py`
Oracle alignment	`python scripts/check_alignment.py`
Error analysis	`python scripts/error_analysis.py runs/E2_a4_lorma`
Comparison table	`python scripts/compare.py`
Figures and tables	`python p2/report/make_figures.py`

To be written by the author — repository link
GitHub: `https://github.com/...`

LLM declaration

To be written by the author — LLM declaration (mandatory)
Fill in the course questionnaire. Describe honestly which parts used an LLM assistant (for example the pipeline and PEFT code, sweep scripts, figures and this report's methods/results text) and which parts are your own (design decisions, hyperparameters, analysis, conclusions).

References

[1] E. Ben Zaken, S. Ravfogel, Y. Goldberg. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. ACL 2022.

[2] H. Bihany, S. Patel, A. Modi. LoRMA: Low-rank multiplicative adaptation for LLMs. Findings of ACL 2025.

[3] G. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[4] N. Houlsby et al. Parameter-efficient transfer learning for NLP. ICML 2019.

[5] E. Hu et al. LoRA: Low-rank adaptation of large language models. ICLR 2022.

[6] J. Lafferty, A. McCallum, F. Pereira. Conditional random fields. ICML 2001.

[7] T.-Y. Lin et al. Focal loss for dense object detection. ICCV 2017.

[8] H. Liu et al. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. NeurIPS 2022.